

Report conversation

in my github project 2 PRs where open. one containing the commits: A, B and C and the second one containing A, B, C and D.

now the first PR with A, B, C was merged by squashing and the second PR now has merge conflicts

It seems like you're facing a common issue in version control systems like Git, especially when working with pull requests (PRs) and the squash merge strategy. Here's what likely happened in your situation:

- 1. Two PRs with Overlapping Commits:** You had two PRs open. The first PR included commits A, B, and C, while the second PR included commits A, B, C, and an additional commit D.
- 2. First PR Squashed and Merged:** The first PR was merged into the main branch by squashing. This means commits A, B, and C were combined into a single new commit on the main branch. This new commit has a different commit ID than the original A, B, and C commits.
- 3. Merge Conflicts in Second PR:** The second PR, which still contains the original commits A, B, C, and D, now has merge conflicts. This is because the main branch has a new squashed commit that differs from the individual A, B, and C commits in the second PR.

To resolve this situation, you can follow these steps:

- 1. Update Your Local Main Branch:** First, make sure your local main branch is up to date with the remote main branch. This will include the new squashed commit.

```
bash
```

[Copy code](#)

```
git checkout main
git pull origin main
```

- 2. Rebase the Second PR:** Next, rebase the second PR onto the updated main branch. This will place your commit D on top of the current state of the main branch.

```
bash
```

```
git checkout second-pr-branch  
git rebase main
```

- 3. Resolve Conflicts:** During the rebase, you may encounter conflicts due to the changes in commits A, B, and C being different from the squashed commit. Resolve these conflicts manually in your code editor.
- 4. Continue the Rebase:** After resolving the conflicts, continue the rebase.

```
bash
```

 Copy code

```
git add .  
git rebase --continue
```

- 5. Force Push the Updated PR:** Once the rebase is complete and all conflicts are resolved, you will need to force push the changes to the remote branch of the second PR.

```
bash
```

 Copy code

```
git push origin second-pr-branch --force
```

By doing this, your second PR will now be based on the current state of the main branch, including the squashed changes from the first PR, and only contain the additional changes from commit D. This should resolve the merge conflicts and allow for a clean merge of the second PR.

what about doing a soft reset instead of rebasing?

Performing a soft reset is indeed another viable option to resolve this situation. The soft reset approach is generally simpler and can be more intuitive if you're not as comfortable with rebasing. Here's how you can do it:

- 1. Update Your Local Main Branch:** First, ensure that your local main branch is up to date with the remote main branch, which includes the new squashed commit.

```
bash
```

 Copy code

```
git checkout main
git pull origin main
```

2. Checkout the Second PR Branch: Switch to the branch of the second PR.

```
bash
```

 Copy code

```
git checkout second-pr-branch
```

3. Perform a Soft Reset: Soft reset the branch to the updated main branch. This will keep all the changes in your working directory but reset the commit history to match the main branch.

```
bash
```

 Copy code

```
git reset --soft main
```

After this step, the changes from commits A, B, C, and D will be unstaged in your working directory.

4. Stage and Commit the Changes: Now, you can stage and commit these changes. This will create a new commit that includes the changes from your original commits A, B, C, and D, but based on the updated main branch.

```
bash
```

 Copy code

```
git add .
git commit -m "Combined changes from A, B, C, and D"
```

5. Force Push the Updated PR: Finally, force push this new commit to the remote branch of the second PR. This is necessary because the history of your branch has changed.

```
bash
```

 Copy code

```
git push origin second-pr-branch --force
```

This approach essentially discards the individual commit history of A, B, C, and D from your PR branch and creates a new single commit that contains all the changes from these commits, but is based on the latest main branch that includes the squashed merge. It's a

cleaner way to integrate your changes without dealing with the intricacies of rebasing and resolving conflicts commit by commit.

since main already contains changes from A, B, C the changes that are remaining should be exclusively D, right?

Yes, you are correct. Since the main branch already contains the changes from commits A, B, and C (through the squashed commit), the remaining changes in your second PR branch should exclusively be from commit D. When you perform a soft reset and then commit, this new commit should ideally only reflect the changes introduced in commit D.

To clarify:

- 1. Soft Reset Keeps D's Changes:** When you perform `git reset --soft main` on your second PR branch, it resets the branch's commit history to match main but keeps the changes from your commits (A, B, C, D) in the working directory.
- 2. Main Already Has A, B, C:** Because the main branch already includes A, B, and C (squashed into one commit), the changes you see in your working directory after the soft reset should only be those that are unique to D.
- 3. Committing Only D's Changes:** When you add and commit these changes, this new commit should effectively only include the changes from D, as the changes from A, B, and C are already in the main branch and wouldn't appear as new changes.

To proceed:

- 1. Soft Reset:** Reset the second PR branch to the main branch.

```
bash
```

 Copy code

```
git reset --soft main
```

- 2. Check Changes:** Before committing, it's a good practice to check the changes that are staged. You should only see the changes from D here.

```
bash
```

 Copy code

```
git status
```

```
git diff --cached
```

3. Commit and Push: Commit these changes and push them to your second PR branch.

```
bash
```

 Copy code

```
git commit -m "Implement changes from commit D"  
git push origin second-pr-branch --force
```

This will update your PR with a new commit that only contains the changes from D, making it ready to be merged into the main branch without conflicts from the previously merged A, B, and C commits.